COLÉGIO DR. CLÓVIS BEVILACQUA

CURSO DE INFORMÁTICA

ALEXANDER MOLANY LONGO
LUCAS RODRIGUES SOARES
PEDRO GALVÃO DE MATOS
YASMIN REGINA DE SOUZA

DOCUMENTAÇÃO TÉCNICA DO SISTEMA SPOT: GESTÃO INTEGRADA DE PROJETOS

Santo André

2026

ALEXANDER MOLANY LONGO
LUCAS RODRIGUES SOARES
PEDRO GALVÃO DE MATOS
YASMIN REGINA DE SOUZA

DOCUMENTAÇÃO TÉCNICA DO SISTEMA SPOT: GESTÃO INTEGRADA DE PROJETOS

Documentação técnica complementar ao Trabalho de Conclusão de Curso SPOT, apresentada ao Curso de Informática do Colégio Dr. Clóvis Bevilacqua.
Orientador: Prof. João Carlos de Souza.

Santo André

2026

RESUMO

Esta documentação técnica do sistema Sistemas de Projetos, Organização e Tarefas (SPOT) registra a arquitetura, as regras de negócio, o modelo de dados, os contratos de API e as condições de operação do SPOT na versão consultada em 10 de novembro de 2026. O levantamento utiliza o código-fonte identificado por commit, migrations, rotas, componentes e evidências automatizadas, sem tratar propostas como recursos entregues. A aplicação emprega Laravel 13, React 18, PostgreSQL e consolidação analítica com Python e pandas. São descritos provisionamento administrativo e ativação por e-mail, controle de permissões, gerente obrigatório, execução de projetos, finanças, agenda, notificações e chat com aprovador escolhido. Também são registrados os modelos de e-mail em três idiomas, o tutorial, os recursos de leitura e a tradução automática opcional de mensagens. A revisão aprovou 256 testes de servidor com 3.487 asserções, inclusive em contêiner, e 36 suítes de navegador, além do build e das auditorias de dependências. A entrega corresponde a código verificado para homologação e não a uma declaração de produção liberada. Credenciais, domínio, integrações reais, responsáveis de projetos antigos e revisão humana de acessibilidade permanecem como condições de aceite. Referências atualizadas, inventários e o link da versão do repositório permitem consultar os artefatos e reproduzir a avaliação.

Palavras-chave: gestão de projetos; aplicação web; controle de acesso; análise de dados; testes de software.

ABSTRACT

This technical documentation records the architecture of SPOT (Sistemas de Projetos, Organização e Tarefas), business rules, data structures, API contracts and operational conditions for the source version examined on November 10, 2026. It draws on the identified commit, migrations, routes, components and automated evidence, without treating planned capabilities as delivered features. The application uses Laravel 13, React 18, PostgreSQL, and analytical processing with Python and pandas. It describes administrative provisioning, email-based activation, permissions, required project ownership, project execution, finances, calendar, notifications and chat with a selected approver. The documentation also covers three-language email templates, the tutorial, reading adjustments and optional automatic message translation. Verification passed 256 server tests with 3,487 assertions, including container execution, and 36 browser suites, together with the frontend build and dependency audits. The delivery represents code verified for acceptance testing, rather than an unconditional production release. Credentials, domain setup, real integrations, legacy ownership records and human accessibility review remain acceptance conditions. Updated references, inventories and a source-version link support consultation and reproducibility.

Keywords: project management; web application; access control; data analysis; software testing.

LISTA DE FIGURAS

Figura 1 — Organização das principais camadas do SPOT.....11
Figura 2 — Solicitação de chat com escolha do aprovador.....17

LISTA DE QUADROS

Quadro 1 — Tecnologias da versão consultada.....10
Quadro 2 — Regras centrais verificadas.....13
Quadro 3 — Grupos de dados.....20
Quadro 4 — Rastreabilidade entre implementação e testes.....21
Quadro 5 — Síntese da verificação da versão final.....23
Quadro 6 — Rotas inventariadas.....30
Quadro 7 — Migrations disponíveis.....35
Quadro 8 — Documentos operacionais complementares.....36

LISTA DE SIGLAS

ABNT — Associação Brasileira de Normas Técnicas
API — Application Programming Interface
CSRF — Cross-Site Request Forgery
CSV — Comma-Separated Values
HTML — Hypertext Markup Language
HTTP — Hypertext Transfer Protocol
HTTPS — Hypertext Transfer Protocol Secure
IA — Inteligência artificial
JSON — JavaScript Object Notation
JWT — JSON Web Token
PHP — PHP: Hypertext Preprocessor
SMTP — Simple Mail Transfer Protocol
SQL — Structured Query Language
TCC — Trabalho de Conclusão de Curso
TI — Tecnologia da informação
WCAG — Web Content Accessibility Guidelines

SUMÁRIO

1 IDENTIFICAÇÃO E ESCOPO....8
2 ORGANIZAÇÃO DOCUMENTAL E NORMALIZAÇÃO....9
3 ARQUITETURA E AMBIENTE TECNOLÓGICO....10
3.1 Componentes e versões....10
3.2 Banco relacional e concorrência....11
3.3 Comunicação e implantação....11
4 REQUISITOS, REGRAS E FUNCIONALIDADES....13
4.1 Requisitos rastreáveis....13
4.2 Identidade, convite e primeiro acesso....15
4.3 Projetos, equipe e execução....15
4.4 Chat e aprovação direcionada....16
4.5 Agenda, notificações e e-mails....17
4.6 Portfólio e processamento analítico....18
4.7 Idiomas, acessibilidade e tutorial....18
4.8 Tradução automática e escopo de IA....18
5 MODELO DE DADOS E CONTRATOS DE API....20
5.1 Vínculos e integridade....20
5.2 Contratos e erros....21
6 VERIFICAÇÃO, RESULTADOS E LIMITES....23
6.1 Cobertura comportamental....23
6.2 Ameaças à validade e critérios de aceite....24
7 IMPLANTAÇÃO E CONTINUIDADE....25
7.1 Preparação e configuração....25
7.2 Validação do servidor de destino....25
7.3 Operação e manutenção....26
7.4 Consulta ao código e reprodução....26
8 SÍNTESE TÉCNICA....27
REFERÊNCIAS....28
APÊNDICE A — INVENTÁRIO DAS ROTAS DA APLICAÇÃO....30
APÊNDICE B — EVOLUÇÃO DO ESQUEMA E ARTEFATOS....35

1 IDENTIFICAÇÃO E ESCOPO

Este relatório técnico complementa a monografia do sistema Sistemas de Projetos, Organização e Tarefas (SPOT) e descreve a versão consultada em 10 de novembro de 2026. A base é o commit 146bd986867ecb25bc75db53c6d1944c6d89ae66, disponível no repositório configurado pela equipe. Foram considerados código, configuração, migrations, rotas, testes e evidências de prontidão; arquivos de pesquisa e versões anteriores permanecem preservados.

As descrições indicam comportamento presente e condições de operação. Não se declara publicação concluída, entrega de inteligência artificial (IA) gerencial, resultado financeiro causado pelo produto ou conformidade integral de acessibilidade. O sistema utiliza PostgreSQL; referências anteriores a SQLite não descrevem a versão atual.

2 ORGANIZAÇÃO DOCUMENTAL E NORMALIZAÇÃO

A apresentação adota folha A4, margens superior/esquerda de 3 cm e inferior/direita de 2 cm, corpo Arial 12, espaçamento de 1,5 e organização em seções. Citações são indiretas no sistema autor-data; referências são alinhadas à esquerda e em espaçamento simples. Resumo, sumário e numeração seguem as normas correspondentes. Não foram inventados dados de aprovação institucional.

Adotam-se as normas da Associação Brasileira de Normas Técnicas (ABNT). As referências consideram ABNT NBR 14724:2024, NBR 6023:2025, NBR 10520:2023, NBR 6024:2012, NBR 6027:2012 e NBR 6028:2021 (Associação Brasileira de Normas Técnicas, 2012a, 2012b, 2021, 2023, 2024, 2025). As bibliotecas UFES e UFABC foram consultadas para confirmar as atualizações e orientações de normalização (Universidade Federal do ABC, [20--]; Universidade Federal do Espírito Santo, 2026).

3 ARQUITETURA E AMBIENTE TECNOLÓGICO

O SPOT organiza interface, interface de programação de aplicações (API), regras de domínio e persistência no mesmo repositório, com responsabilidades separadas. O navegador apresenta o estado e envia requisições; o servidor valida e autoriza cada operação; classes de domínio e modelos implementam invariantes e transações. A integração não depende de um servidor de inferência para executar as funções centrais (Longo *et al.*, 2026c).

3.1 Componentes e versões

A documentação oficial do Laravel descreve o ciclo de requisição e os serviços de uma aplicação PHP: Hypertext Preprocessor (PHP); o React estrutura a interface em componentes, enquanto o Vite fornece o servidor de desenvolvimento e a construção dos recursos estáticos (Laravel, 2026; Meta Platforms, [20--]; Vite, [20--]). As versões do produto são obtidas dos manifestos, lockfiles e verificações do projeto, não da versão mais recente anunciada nos sites.

As informações correspondentes estão organizadas no Quadro 1.

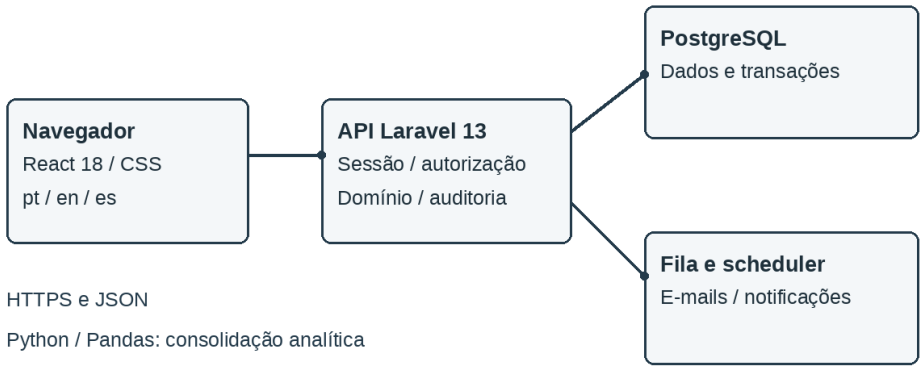
Quadro 1 — Tecnologias da versão consultada

Camada	**Tecnologia**	**Aplicação**
Servidor	PHP 8.3; Laravel 13	Rotas, validação, autorização, transações, filas e correio.
Interface	React 18.3.1; Vite 8.3.5	Componentes, estado, navegação da aplicação e compilação dos assets.
Persistência	PostgreSQL; imagem PostgreSQL 17	Banco relacional único; SQLite não é o banco da versão final.
Analytics	Python 3; pandas 2.3.3 na imagem verificada	Consolidação de indicadores; requirements.txt admite pandas >=2.2 e <3.0.
Editor do chat	Tiptap / ProseMirror	Documento estruturado, listas, links e atalhos, com validação no servidor.
Autenticação	Sessão/Sanctum e JWT	Sessão da aplicação principal; JWT nas rotas de compatibilidade e estoque.
Operação	Docker Compose; FrankenPHP/Caddy	Serviços web, worker, scheduler e PostgreSQL; certificados dependem do domínio real.

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).

A representação correspondente é apresentada na Figura 1.

Figura 1 — Organização das principais camadas do SPOT



Fonte: Elaboração própria a partir do código-fonte (Longo *et al.*, 2026c).

3.2 Banco relacional e concorrência

O PostgreSQL é a persistência única da versão final. Referências a SQLite na monografia preliminar descrevem um estado anterior e foram retiradas da descrição do sistema atual. As migrations definem evolução de tabelas e relacionamentos; transações e bloqueios de linha protegem decisões, movimentações e alterações de equipe. A documentação PostgreSQL fundamenta o uso de integridade e controle transacional (PostgreSQL Global Development Group, 2026).

Na alteração de equipe, um bloqueio no registro do projeto serializa operações que poderiam remover simultaneamente os últimos responsáveis. No chat, o par de usuários é representado por uma conversa e os pedidos pendentes são conferidos sob transação. Esses controles sustentam o comportamento verificado nos testes; não constituem, por si, um ensaio de escalabilidade (Longo *et al.*, 2026c).

3.3 Comunicação e implantação

A aplicação principal utiliza JavaScript Object Notation (JSON) e rotas versionadas sob /api/v1, com sessão, cookies e proteção Cross-Site Request Forgery (CSRF) nas operações mutáveis. A compatibilidade por token e o estoque utilizam JSON Web Token (JWT) em suas rotas. OAuth conecta provedores

externos; sua homologação requer credenciais e permissões reais. Falhas de validação são apresentadas próximas aos formulários e não devem expor Structured Query Language (SQL), traces ou segredos.

O Docker Compose descreve serviços e volumes da aplicação (Docker, 2026). O empacotamento do SPOT inclui web, worker e scheduler, além do PostgreSQL. E-mails usam a fila emails; atividades e limpezas periódicas são agendadas dentro do processo PHP. O arquivo .env e dados de usuários não integram a imagem (Longo *et al.*, 2026c).

4 REQUISITOS, REGRAS E FUNCIONALIDADES

As funcionalidades são descritas por operações, critérios de acesso e condições de funcionamento. As regras estabelecem as responsabilidades dos perfis e os controles aplicados pelo servidor.

4.1 Requisitos rastreáveis

Os requisitos foram derivados das necessidades registradas pela equipe e refinados por comportamento observável no código. Requisito funcional descreve uma operação ou restrição; requisito não funcional trata de qualidades e condições de uso. A entrega inclui autenticação, governança, projetos, tarefas, horas, finanças, comunicação e operação, com rastreabilidade entre regra, componente e teste (Longo *et al.*, 2026c).

As informações correspondentes estão organizadas no Quadro 2.

Quadro 2 — Regras centrais verificadas

Identificação	Tema	Regra
RN01	Provisionamento	Somente usuário com alçada administrativa provisiona conta e gera link de ativação; não há solicitação pública de primeiro acesso.
RN02	Ativação	O e-mail deve corresponder à conta do token; o nome não precisa coincidir. Token deve estar válido e não utilizado; senha precisa de confirmação.
RN03	Reenvio	Uma transição de outro estado para aguardando ativação gera novo link e agenda e-mail; editar uma conta já pendente não repete o envio.
RN04	Projeto	Gerente do projeto obrigatório na criação v1; o responsável entra como manager na equipe. Não se permite remover ou rebaixar o último gerente elegível da equipe.
RN05	Visibilidade	Empresa atual, perfis, participação e exceções de permissão limitam os dados e as operações. Interface não substitui autorização do servidor.

Identificação	Tema	Regra
RN06	Chat direto	Exige vínculo ativo de analyst ou manager dos dois usuários em um mesmo projeto, ou autorização específica válida para o par.
RN07	Pedido de chat	Pessoa, motivo e aprovador são obrigatórios. O aprovador é administrador ou gerente administrador ativo da mesma empresa, diferente do solicitante, inclusive fora do projeto.
RN08	Decisão	Novo pedido vai apenas ao aprovador escolhido; só ele aprova/recusa. Uma pendência por par impede duplicação, inclusive inversa. Pedidos antigos sem destinatário mantêm a compatibilidade.
RN09	Restrições do chat	Convidados e contas inativas não usam o chat. Aprovar não concede leitura de conversas de terceiros. Revogação preserva histórico.
RN10	Mensagens	Apenas autor edita até 24 horas do envio original; versões concorrentes são verificadas. Original e revisões são preservados.
RN11	Tradução	Bandeiras pt/en/es escolhem apenas destino de tradução da mensagem; original preservado. Serviço externo configurado é necessário.
RN12	Recuperação	Intervalo de 60 segundos entre gerações para o mesmo e-mail; link vale 60 minutos. Nova geração substitui a anterior, sem divulgar existência da conta.
RN13	Projetos finalizados	Restrições a alterações e preservação do histórico seguem os endpoints e permissões aplicáveis.
RN14	Despesas e tarifas	Despesas passam pelo fluxo de revisão; custos e preços usam tarifas e vigências, sem sobreposições indevidas.
RN15	Notificações	Preferências, canal, frequência e silêncio condicionam entrega; processamento em fila e deduplicação evitam reenvios indevidos.

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).

4.2 Identidade, convite e primeiro acesso

O primeiro acesso é iniciado por provisionamento administrativo. O link utiliza token com hash persistido, validade e controle de uso. A identificação da ativação compara somente o e-mail com a conta associada ao convite, desconsiderando caixa e espaços nas extremidades; não exige que o nome digitado coincida com o cadastro. O nome administrativo é preservado. Quando o e-mail diverge, a interface orienta: Por gentileza, insira o e-mail correspondente ao do convite enviado! A validade do token e a confirmação da senha continuam obrigatórias.

Uma nova entrada da conta em aguardando ativação revoga links anteriores ainda não usados, cria outro token e agenda a mensagem após confirmar a transação. A validade da ativação administrativa é de 48 horas. Edição de cargo ou perfil de uma conta já pendente não é um novo evento de reenvio. A recuperação de senha é um fluxo diferente: o link vale 60 minutos e há intervalo de 60 segundos entre gerações para a mesma conta (Longo *et al.*, 2026c).

4.3 Projetos, equipe e execução

O cadastro v1 exige gerente do projeto elegível. O usuário selecionado entra na equipe como manager, sem duplicação se também constar na seleção inicial. Equipes, atribuições e remoções de membros preservam ao menos um gerente ativo elegível, sincronizando o campo de referência quando necessário. Perfis de administrador, gerente administrador e gerente podem ser elegíveis para a função, conforme as regras do servidor.

Projetos concentram contrato, situação, datas, membros e planejamento. Tarefas oferecem visualizações de trabalho e apontamentos; tarifas possuem vigência, e despesas passam por revisão. Finalização e reabertura seguem autorização e preservação do histórico. O módulo de documentos mantém suas regras de privacidade: associar um arquivo a projeto ou tarefa não equivale a compartilhá-lo com toda a equipe (Longo *et al.*, 2026c).

4.4 Chat e aprovação direcionada

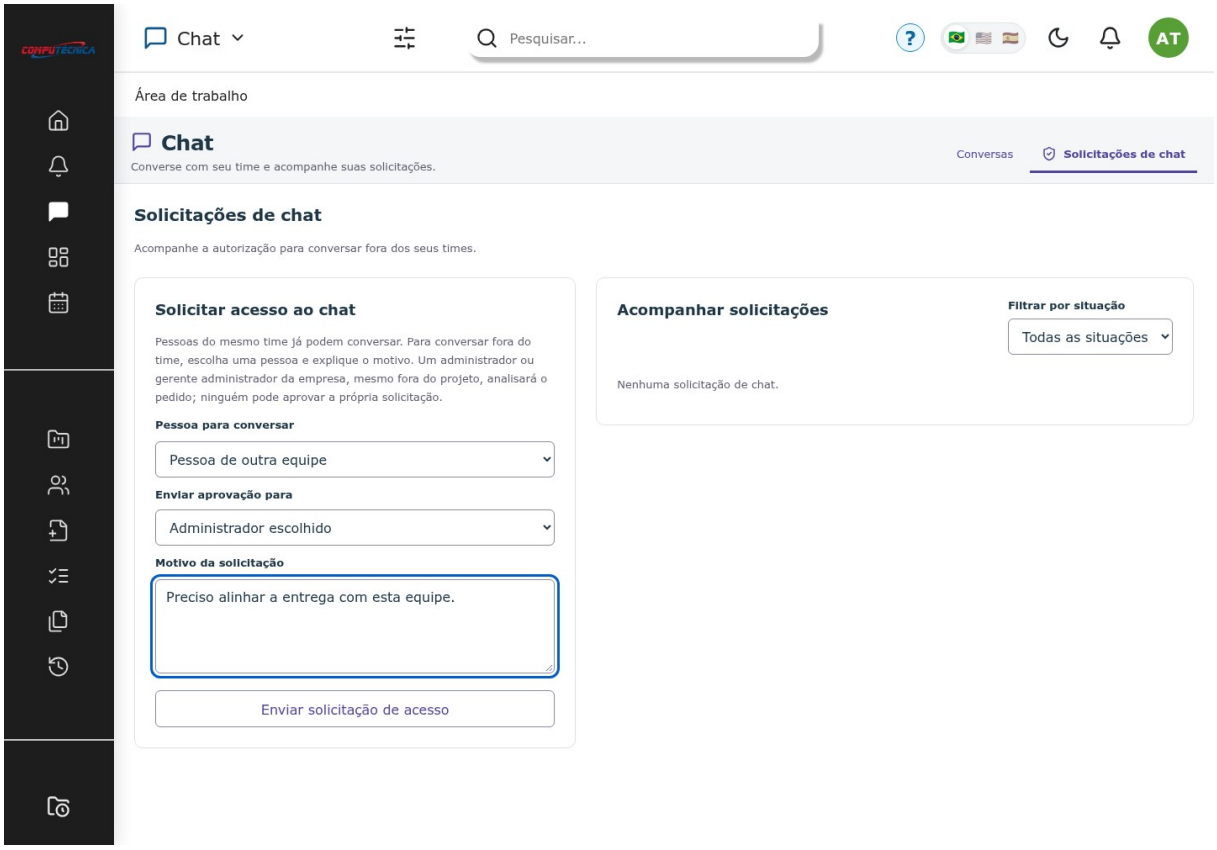
O chat é individual e restrito à empresa atual. Há acesso direto quando os dois usuários possuem participação ativa, não revogada, com papel analyst ou manager em um mesmo projeto. Fora desse vínculo, o formulário solicita pessoa, motivo e o destinatário em Enviar aprovação para. A lista contém administradores e gerentes administradores ativos e elegíveis da mesma empresa, excluindo o solicitante; podem estar fora do projeto.

O campo assigned_reviewer_id registra quem recebeu o novo pedido; reviewer_id registra quem decidiu. Somente o escolhido recebe o aviso e pode aprovar ou recusar. A autorização vale para o par, e não libera a leitura de conversas de terceiros. Pedidos antigos sem atribuição mantêm compatibilidade com o fluxo anterior. Uma pendência por par impede duplicação, inclusive em sentido inverso; pedidos recusados ou revogados permitem nova tentativa conforme a elegibilidade (Longo *et al.*, 2026c).

Apenas o autor pode editar uma mensagem, até 24 horas do envio original. As revisões são preservadas e uma versão é conferida para evitar sobrescrita concorrente. Anexos usam acesso autorizado e descrições de imagem podem ser informadas. O editor fornece formatação e atalhos; Enter envia e Ctrl + Shift + Enter insere nova linha. A existência de autorização de revisão não concede acesso ao conteúdo de uma conversa alheia.

A representação correspondente é apresentada na Figura 2.

Figura 2 — Solicitação de chat com escolha do aprovador



Fonte: Interface do build local, com massa sintética e API simulada; captura de teste, 10 nov. 2026.

4.5 Agenda, notificações e e-mails

A agenda reúne eventos internos e conexões externas disponíveis. Grupos, participantes, fuso e detalhes são validados. As notificações consideram preferências por canal, silêncio, frequência e deduplicação. Resumos diários ou semanais e envios imediatos são processados em lotes e filas, com tratamento de falha e nova tentativa (Longo *et al.*, 2026c).

Ativação, convite, recuperação e notificações compartilham o layout de correio: cabeçalho Spot, título, ação, link alternativo e rodapé, com alternativa em texto simples. Textos de estrutura possuem pt/en/es; dados escritos por pessoas são preservados e escapados no HTML. Disponibilidade de Simple Mail Transfer Protocol (SMTP), reputação do remetente e apresentação em clientes de e-mail reais exigem homologação separada.

4.6 Portfólio e processamento analítico

Os relatórios consolidam situação, custos, receitas, margens, horas e despesas, usando processos Python com pandas. A biblioteca fornece estruturas e operações de análise de dados (Pandas Development Team, 2026). No SPOT, os cálculos dependem dos registros e permissões existentes; falta de tarifa ou base de cálculo não é convertida automaticamente em evidência de lucro ou desempenho.

Painéis permitem indicadores e diferentes visualizações, agregações e exportação CSV. Resultados parciais são identificados. Uma falha do detalhamento financeiro é tratada localmente sem cobrir todos os outros indicadores com uma tela global de erro. O resultado financeiro é gerencial, depende da qualidade dos dados e não equivale a demonstração contábil auditada (Longo *et al.*, 2026c).

4.7 Idiomas, acessibilidade e tutorial

A interface dispõe de português, inglês e espanhol, ajustes de fonte, escala e tema, indicação de foco e respeito à redução de movimento e cores forçadas. O objetivo de acessibilidade é Web Content Accessibility Guidelines (WCAG) 2.2 nível AA, referência que abrange diferentes necessidades de uso; resultados automatizados não substituem avaliação humana nem certificam conformidade integral (World Wide Web Consortium, 2024).

O tutorial é opcional e seleciona as etapas conforme permissões e disponibilidade do chat. Explica navegação, projetos, execução, segurança e solicitações, sem enviar formulários de negócio. A mudança de idioma dentro do passeio não grava alterações na conta. A posição dos destaques é verificada para telas disponíveis e áreas restritas. Nomes e conteúdos pessoais usam marcação para impedir tradução automática da interface (Longo *et al.*, 2026c).

4.8 Tradução automática e escopo de IA

As bandeiras no cabeçalho da conversa escolhem apenas o destino da tradução da mensagem, sem alterar o idioma global. O texto traduzido é apresentado no balão, identificado como automático, com acesso ao original. A

integração opcional usa o endpoint de tradução do LibreTranslate (LibreTranslate, [20--]). Credenciais e endereço do serviço permanecem no servidor.

É necessário distinguir tradução automática de agentes de planejamento ou geração de conteúdo. A versão final não demonstra um agente generativo para estimar projetos, redistribuir equipes ou tomar decisões gerenciais. Automação de notificações, regras determinísticas e consolidação com pandas também não são, por si, implementação de inteligência artificial. A pesquisa de tecnologias de IA é referência de possibilidades, não prova de entrega dessas funções.

5 MODELO DE DADOS E CONTRATOS DE API

O esquema é descrito por migrations versionadas. As entidades agrupam identidade, organização, trabalho, finanças, comunicação e operação. O quadro a seguir é um mapa de domínio; o apêndice de migrations e os arquivos do repositório são a referência para nomes, tipos, nulabilidade, índices e chaves exatos (Longo *et al.*, 2026c).

As informações correspondentes estão organizadas no Quadro 3.

Quadro 3 — Grupos de dados

Domínio	**Entidades principais**	**Responsabilidade**
Identidade e acesso	users, profiles, permissions, company_user_profiles, user_permission_overrides	Contas, empresa atual, perfis e exceções.
Organização e projetos	companies, organizations, clients, projects, project_members	Responsáveis, contrato, equipe e vínculos.
Execução e finanças	tasks, work_logs, project_expenses, analyst_project_rates, project_taxes	Atividades, horas, despesas, tarifas e composição financeira.
Convites e ativação	project_invitations, user_activation_tokens	Tokens armazenados como hash, validade e uso/revogação.
Chat	chat_conversations, chat_access_requests, chat_messages, chat_message_revisions, chat_attachments	Par de usuários, aprovador escolhido/decisor, versões e anexos privados.
Agenda e notificações	calendar_events, calendar_deliveries, calendar_event_changes, inbox_items, notification_deliveries, notification_mail_batches	Eventos, participantes/grupos, alertas, entregas e resumos.
Operação	jobs, failed_jobs, audit_logs e tabelas de sessão/cache	Fila, falhas, rastreabilidade e suporte ao runtime.
Estoque	categories, products, stock_movements	Camada de compatibilidade com autenticação JWT e movimentação atômica.

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).

5.1 Vínculos e integridade

Uma empresa limita as contas e os projetos visíveis. Projetos relacionam responsáveis e membros; tarefas, horas e despesas referenciam o contexto de execução. Conversas representam um par ordenado de usuários, enquanto

solicitações, mensagens, revisões e anexos mantêm relações separadas. Tokens de acesso não são armazenados em texto simples. O responsável escolhido para aprovação não é confundido com o usuário que concluiu a decisão.

A preservação do histórico é parte do domínio: revogar autorização não remove mensagens; finalização limita operações sem apagar a trajetória do projeto; exclusão de registros com dependências segue restrições e regras aplicáveis. A normalização lógica evita repetir a mesma credencial, perfil ou estado em entidades sem necessidade, embora haja snapshots controlados para rastreabilidade de valores e decisões.

5.2 Contratos e erros

O inventário exportado pelo Laravel contém 170 rotas no ambiente consultado, incluindo infraestrutura e framework. Esse total não representa 170 funcionalidades de negócio. O apêndice discrimina rotas da aplicação e o manifesto preserva o critério de extração.

Respostas de validação usam a estrutura errors e os códigos Hypertext Transfer Protocol (HTTP) compatíveis com o problema. Requisições sem autorização, recursos não visíveis, conflito de estado e indisponibilidade têm tratamentos distintos. O cliente de sessão inicializa CSRF antes das mutações e só publica eventos de atualização após sucesso; logout não dispara recarga automática de dados privados. Validações críticas continuam no servidor (Longo *et al.*, 2026c).

As informações correspondentes estão organizadas no Quadro 4.

Quadro 4 — Rastreabilidade entre implementação e testes

Área	**Artefatos**	**Verificação**
Identidade	app/Http/Controllers/Api/ AdminUserController.php; FirstAccessController.php	AdministrativeProvisioningTest; ActivationStatusEmailTest
Projetos/equipe	ProjectController.php; TeamController.php; ProjectTeamManagers.php	RequiredProjectManagerTest; InitialProjectTeamTest
Chat e aprovação	app/Domain/Chat/ ChatAccess.php; ChatController.php; ChatPage.jsx	ChatTest; scripts/test-chat.mjs
Editor e original	ChatRichEditor.jsx; ChatFormattedText.jsx; chatFormatting.js	scripts/test-chat-formatting.mjs

Área	Artefatos	Verificação
Correio e idioma	app/Mail; app/Domain/Mail; resources/views/mail	LocalizedResetMailTest; StandardizedMailTest
Notificações/agenda	NotificationCenter; SendNotificationBatch; CalendarController	NotificationDeliveryTest; testes de agenda
Acessibilidade/tutorial	useModalFocus.js; usePopoverPosition.js; tutorial/content.js	test-accessibility.mjs; test-public-accessibility.mjs; test-tutorial.mjs
Analytics	ProjectAnalyticsController.php; scripts/project_analytics.py; PortfolioChart.jsx	FinancialReportTest; test-portfolio.mjs
Implantação	Dockerfile; docker-compose.yml; routes/console.php	ProductionSchedulerTest; verify-container-tests.py; check-production.php

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).

6 VERIFICAÇÃO, RESULTADOS E LIMITES

A evidência técnica corresponde à execução registrada em 10 de novembro de 2026. O código consultado foi identificado por commit e acompanhado de logs e inventários. Resultados antigos de 52 testes e 262 asserções pertencem a uma edição anterior e não descrevem a versão final. A quantidade de testes não deve ser interpretada como quantidade de funcionalidades nem como cobertura total (Longo *et al.*, 2026c).

As informações correspondentes estão organizadas no Quadro 5.

Quadro 5 — Síntese da verificação da versão final

Verificação	**Resultado**	**Limite**
PHPUnit local	256 testes; 3.487 asserções; aprovado	Schema spot_testing; dados sintéticos.
PHPUnit em contêiner	256 testes; 3.487 asserções; aprovado	PHP 8.3 e PostgreSQL descartável; serviços externos simulados.
Navegador	36 suítes aprovadas	Chromium; não representa todos os navegadores.
Responsividade	323 verificações aprovadas	320–1440 px; orientação vertical/horizontal; leitura 100%/120%.
Tutorial	756 verificações; sem erros	Regras de navegação, permissões, idioma e ausência de mutações de negócio.
Idiomas	144 verificações; sem falhas ou apontamentos	Cobertura dos percursos automatizados; não certifica toda expressão possível.
Chat/editor	170 verificações; formatação 3 × 100	Inclui atalhos, destinatário e preservação do original.
Correio	36 prévias de layout	Seis variantes; três idiomas; duas larguras; sem destinatários reais.
Build e dependências	Build e auditorias sem avisos na rodada final	Situação verificada na data de corte, não garantia permanente.
Docker	Imagem construída e rotas públicas com HTTP 200	Não é ensaio de certificado, rede de produção ou SMTP real.

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).

6.1 Cobertura comportamental

Os testes incluem autorização, isolamento de empresa, identidade, ativação e reenvio, recuperação, mudanças de projeto, gerente obrigatório, acesso a

documentos, despesas, tarifas e notificações. No chat, verificam participação, aprovação selecionada, autoaprovação proibida, duplicatas, edição, versões, anexos e revogação. Nos testes de correio, a entrega usa transporte em memória; não há envio para destinatários reais.

Na interface, foram usados nomes longos, tela estreita, orientação horizontal, tema, escala de leitura, fluxo por teclado, foco em modais, retorno de foco e tradução. O teste do editor foi repetido após corrigir a disputa de foco. O passeio do tutorial foi percorrido segundo permissões, inclusive quando relatórios não estão disponíveis (Longo *et al.*, 2026c).

6.2 Ameaças à validade e critérios de aceite

A avaliação automatizada comprova apenas os cenários exercitados no ambiente descrito. Não houve comprovação empírica de redução de custo, aumento de produtividade ou satisfação após implantação. Não foram realizados testes de carga com volume real, auditoria ofensiva completa, homologação com todos os clientes de e-mail ou avaliação humana integral com leitor de tela.

Para aceite operacional, devem ser confirmados domínio e Hypertext Transfer Protocol Secure (HTTPS), segredos, banco e migrations, armazenamento, backup e restauração, workers e scheduler. Recursos Google, Microsoft, SMTP e tradução dependem das permissões e serviços reais. A existência de uma integração no código não equivale à autorização de um tenant nem à disponibilidade do provedor. Não se declara conformidade integral WCAG ou liberação incondicional para produção.

7 IMPLANTAÇÃO E CONTINUIDADE

A entrada em operação depende do ambiente de destino e da homologação das integrações. Esta seção organiza os requisitos de configuração, validação e manutenção.

7.1 Preparação e configuração

A implantação proposta usa Docker Compose com serviços web, worker, scheduler e PostgreSQL. O domínio precisa apontar para o servidor e o proxy depende de condições reais para emitir HTTPS. O arquivo .env.production é preenchido a partir do modelo; APP_KEY e JWT_SECRET devem ser configuradas e preservadas nas atualizações. A chave JWT é necessária para o estoque e a autenticação por token. Segredos não integram o Git nem os documentos de consulta.

O build inclui os arquivos estáticos, dependências PHP de produção e ambiente Python para os relatórios. Backups, uploads locais, evidências de testes, .env e public/hot são excluídos do artefato. Uploads reais permanecem nos volumes de armazenamento. Migrations são aplicadas após backup e revisão; o seed da conta inicial não deve ser executado como rotina de atualização (Longo *et al.*, 2026c).

7.2 Validação do servidor de destino

O verificador scripts/check-production.php é somente leitura: verifica ambiente, debug, origem HTTPS, cookies, chaves, correio configurado, existência de assets, diretórios graváveis e conexão/migrations. Sua saída não expõe credenciais nem envia mensagens. Mesmo com saída zero, é necessária a homologação externa e a conferência dos responsáveis nos dados transferidos.

A revisão local identificou três projetos antigos sem o campo de gerente do projeto preenchido, embora todos possuíssem gerente ativo na equipe. Não houve escolha automática de responsáveis. Esse achado refere-se ao conjunto local revisado e deve ser reavaliado na base que será implantada; não é uma característica fixa do software.

7.3 Operação e manutenção

Workers devem consumir emails e default, e o scheduler deve ser executado por minuto. Há controles de nova tentativa e consulta de falhas. Backups devem incluir banco e arquivos, com restauração efetivamente testada. Uma mudança de chave pode afetar dados e jobs criptografados; atualizar a aplicação não exige substituir as chaves existentes.

A manutenção deve preservar rastreabilidade entre alteração, teste, documentação e release. A equipe ainda precisa concluir a validação humana de acessibilidade, os testes com provedores reais e a observação sob carga de uso. O custo de infraestrutura, correio, manutenção e suporte continua existindo; ausência de uma assinatura de software de terceiros não significa operação sem custos.

7.4 Consulta ao código e reprodução

O repositório abaixo permite consultar a estrutura, as regras, as migrations e os testes. A referência de versão evita atribuir à monografia funcionalidades introduzidas posteriormente. A data de acesso foi registrada e a disponibilidade de consulta depende das permissões configuradas no GitHub.

Repositório do SPOT: https://github.com/iamnotlittleprince/SPOT

Versão do código analisada: 146bd986867ecb25bc75db53c6d1944c6d89ae66

O commit consultado corresponde ao HEAD remoto confirmado durante a revisão. A documentação gerada nesta atualização pode ainda estar como alteração local; ela não é considerada publicada apenas por existir na pasta do projeto. O manifesto de fontes e a pasta de evidências registram a procedência da edição.

8 SÍNTESE TÉCNICA

O código e os percursos automatizados estão verificados para homologação, com correções de dependências, isolamento de testes, tutorial, editor e empacotamento. A decisão de liberar produção depende de configurar o ambiente final, conferir os responsáveis antigos e validar integrações e operação reais. O roteiro em docs/implantacao-docker.md e o verificador de produção auxiliam essa etapa sem expor segredos nem enviar mensagens por conta própria.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520:2023**. Informação e documentação — Citações em documentos — Apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724:2024**. Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro: ABNT, 2024. Versão corrigida em 1 abr. 2025.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023:2025**. Informação e documentação — Referências — Elaboração. Rio de Janeiro: ABNT, 2025.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6024:2012**. Informação e documentação — Numeração progressiva das seções de um documento — Apresentação. Rio de Janeiro: ABNT, 2012a.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6027:2012**. Informação e documentação — Sumário — Apresentação. Rio de Janeiro: ABNT, 2012b.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6028:2021**. Informação e documentação — Resumo, resenha e recensão — Apresentação. Rio de Janeiro: ABNT, 2021.

DOCKER. **Docker Compose**. Docker, c2026. Disponível em: https://docs.docker.com/compose/. Acesso em: 10 nov. 2026.

LARAVEL. **Laravel 13.x documentation**. Laravel, c2026. Disponível em: https://laravel.com/docs/13.x. Acesso em: 10 nov. 2026.

LIBRETRANSLATE. **Translate text**. LibreTranslate, [20--]. Disponível em: https://docs.libretranslate.com/api/operations/translate/. Acesso em: 10 nov. 2026.

LONGO, Alexander Molany; SOARES, Lucas Rodrigues; MATOS, Pedro Galvão de; SOUZA, Yasmin Regina de. **SPOT**: sistemas de projetos, organização e tarefas. 2026c. Código-fonte. Versão consultada: commit 146bd986867ecb25bc75db53c6d1944c6d89ae66. Disponível em: https://github.com/iamnotlittleprince/SPOT/tree/146bd986867ecb25bc75db53c6d1944c6d89ae66. Acesso em: 10 nov. 2026.

META PLATFORMS. **React 18**: quick start. Meta, [20--]. Disponível em: https://18.react.dev/learn. Acesso em: 10 nov. 2026.

PANDAS DEVELOPMENT TEAM. **pandas 2.3.3 documentation**. pandas, c2026. Disponível em: https://pandas.pydata.org/pandas-docs/version/2.3/. Acesso em: 10 nov. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**. PostgreSQL, c2026. Disponível em: https://www.postgresql.org/docs/17/. Acesso em: 10 nov. 2026.

UNIVERSIDADE FEDERAL DO ABC. Sistema de Bibliotecas. **Normalização**. Santo André: UFABC, [20--]. Disponível em: https://portal.biblioteca.ufabc.edu.br/servicos/normalizacao. Acesso em: 10 nov. 2026.

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO. Sistema Integrado de Bibliotecas. **Normalização**. Vitória: UFES, 2026. Atualizado em 28 out. 2026. Disponível em: https://biblioteca.ufes.br/manuais-de-normalizacao. Acesso em: 10 nov. 2026.

VITE. **Getting started**. Vite, [20--]. Disponível em: https://vite.dev/guide/. Acesso em: 10 nov. 2026.

WORLD WIDE WEB CONSORTIUM. **Web Content Accessibility Guidelines (WCAG) 2.2**. W3C Recommendation, 12 dez. 2024. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 10 nov. 2026.

APÊNDICE A — INVENTÁRIO DAS ROTAS DA APLICAÇÃO

O quadro foi gerado a partir de artisan route:list --json. Rotas de infraestrutura fora dos prefixos relacionados à aplicação não são reproduzidas; o inventário integral permanece no arquivo de fontes documentais.

As informações correspondentes estão organizadas no Quadro 6.

Quadro 6 — Rotas inventariadas

Método	**URI**	**Nome**
GET|HEAD	accept-invitation/{token}	—
POST	api/auth/login	—
POST	api/auth/logout	—
GET|HEAD	api/auth/me	—
GET|HEAD	api/categories	categories.index
POST	api/categories	categories.store
PUT|PATCH	api/categories/{category}	categories.update
DELETE	api/categories/{category}	categories.destroy
GET|HEAD	api/dashboard	—
GET|HEAD	api/movements	—
POST	api/movements	—
GET|HEAD	api/products	products.index
POST	api/products	products.store
GET|HEAD	api/products/{product}	products.show
PUT|PATCH	api/products/{product}	products.update
DELETE	api/products/{product}	products.destroy
GET|HEAD	api/projects	projects.index
POST	api/projects	projects.store
GET|HEAD	api/projects/{project}	projects.show
PUT|PATCH	api/projects/{project}	projects.update
DELETE	api/projects/{project}	projects.destroy
GET|HEAD	api/tasks	tasks.index
POST	api/tasks	tasks.store
PUT|PATCH	api/tasks/{task}	tasks.update
DELETE	api/tasks/{task}	tasks.destroy
GET|HEAD	api/v1/admin/users	v1.admin.users.index
POST	api/v1/admin/users	v1.admin.users.store
PATCH	api/v1/admin/users/{user}	v1.admin.users.update
DELETE	api/v1/admin/users/{user}	v1.admin.users.destroy
PATCH	api/v1/auth/locale	v1.
POST	api/v1/auth/login	v1.
POST	api/v1/auth/logout	v1.
GET|HEAD	api/v1/auth/me	v1.

Método	URI	Nome
GET|HEAD	api/v1/auth/me/projects	v1.
GET|HEAD	api/v1/auth/timezones	v1.
POST	api/v1/auth/two-factor/challenge	v1.
GET|HEAD	api/v1/calendar	v1.
POST	api/v1/calendar/events	v1.
PATCH	api/v1/calendar/events/{calendarEvent}	v1.
DELETE	api/v1/calendar/events/{calendarEvent}	v1.
PATCH	api/v1/calendar/events/{eventId}/agendas	v1.
GET|HEAD	api/v1/categories	v1.categories.index
POST	api/v1/categories	v1.categories.store
PUT|PATCH	api/v1/categories/{category}	v1.categories.update
DELETE	api/v1/categories/{category}	v1.categories.destroy
GET|HEAD	api/v1/chat	v1.
GET|HEAD	api/v1/chat/attachments/{attachment}/download	v1.
GET|HEAD	api/v1/chat/attachments/{attachment}/preview	v1.
PATCH	api/v1/chat/messages/{message}	v1.
GET|HEAD	api/v1/chat/messages/{message}/revisions	v1.
POST	api/v1/chat/messages/{message}/translate	v1.
POST	api/v1/chat/requests/{chatRequest}/decision	v1.
POST	api/v1/chat/requests/{chatRequest}/revoke	v1.
GET|HEAD	api/v1/chat/users/{user}/messages	v1.
POST	api/v1/chat/users/{user}/messages	v1.
POST	api/v1/chat/users/{user}/requests	v1.
GET|HEAD	api/v1/dashboard	v1.
GET|HEAD	api/v1/documents	v1.
POST	api/v1/documents	v1.
PATCH	api/v1/documents/{document}	v1.
DELETE	api/v1/documents/{document}	v1.
GET|HEAD	api/v1/documents/{document}/download	v1.
PUT	api/v1/expenses/{expense}	v1.
DELETE	api/v1/expenses/{expense}	v1.

Método	URI	Nome
POST	api/v1/expenses/{expense}/review	v1.expenses.review
POST	api/v1/first-access/{token}	v1.first-access.activate
GET|HEAD	api/v1/history	v1.
GET|HEAD	api/v1/home-dashboard	v1.
PATCH	api/v1/home-dashboard/tasks/{task}/priority	v1.
PATCH	api/v1/home-dashboard/tasks/{task}/toggle	v1.
GET|HEAD	api/v1/inbox	v1.
DELETE	api/v1/inbox	v1.
PATCH	api/v1/inbox/read-all	v1.
DELETE	api/v1/inbox/{inboxItem}	v1.
PATCH	api/v1/inbox/{inboxItem}/read	v1.
PATCH	api/v1/inbox/{inboxItem}/restore	v1.
POST	api/v1/invitations/{invitation}/revoke	v1.invitations.revoke
GET|HEAD	api/v1/invitations/{token}	v1.invitations.show
POST	api/v1/invitations/{token}/accept	v1.invitations.accept
GET|HEAD	api/v1/management/access	v1.
GET|HEAD	api/v1/management/directory	v1.
POST	api/v1/management/directory/import	v1.
GET|HEAD	api/v1/management/parameters/{type}	v1.
POST	api/v1/management/parameters/{type}	v1.
PUT	api/v1/management/parameters/{type}/{id}	v1.
DELETE	api/v1/management/parameters/{type}/{id}	v1.
GET|HEAD	api/v1/management/settings	v1.
PUT	api/v1/management/settings	v1.
GET|HEAD	api/v1/management/users/{user}/permissions	v1.
PUT	api/v1/management/users/{user}/permissions	v1.
GET|HEAD	api/v1/movements	v1.
POST	api/v1/movements	v1.
GET|HEAD	api/v1/notification-preferences	v1.
PUT	api/v1/notification-preferences	v1.
DELETE	api/v1/notification-preferences	v1.
GET|HEAD	api/v1/parameters	v1.parameters.index
GET|HEAD	api/v1/products	v1.products.index
POST	api/v1/products	v1.products.store

Método	URI	Nome
GET|HEAD	api/v1/products/{product}	v1.products.show
PUT|PATCH	api/v1/products/{product}	v1.products.update
DELETE	api/v1/products/{product}	v1.products.destroy
GET|HEAD	api/v1/project-access	v1.
POST	api/v1/project-access	v1.
GET|HEAD	api/v1/project-access/meetings	v1.
PATCH	api/v1/project-access/projects/{project}/data	v1.
POST	api/v1/project-access/{accessRequest}/decision	v1.
POST	api/v1/project-access/{accessRequest}/revoke	v1.
GET|HEAD	api/v1/projects	v1.projects.index
POST	api/v1/projects	v1.projects.store
DELETE	api/v1/projects/{project}	v1.projects.governed-destroy
GET|HEAD	api/v1/projects/{project}	v1.projects.show
PUT|PATCH	api/v1/projects/{project}	v1.projects.update
GET|HEAD	api/v1/projects/{project}/configuration	v1.projects.configuration.show
GET|HEAD	api/v1/projects/{project}/expenses	v1.projects.expenses.index
POST	api/v1/projects/{project}/expenses	v1.projects.expenses.store
POST	api/v1/projects/{project}/finalize	v1.projects.finalize
GET|HEAD	api/v1/projects/{project}/financial-result	v1.projects.financial-result
POST	api/v1/projects/{project}/invitations	v1.projects.invitations.store
POST	api/v1/projects/{project}/members	v1.projects.members.store
DELETE	api/v1/projects/{project}/members/{user}	v1.
POST	api/v1/projects/{project}/rates	v1.projects.rates.store
POST	api/v1/projects/{project}/reopen	v1.projects.reopen
GET|HEAD	api/v1/projects/{project}/resources	v1.
PUT	api/v1/projects/{project}/taxes	v1.projects.taxes.replace
GET|HEAD	api/v1/projects/{project}/work-logs	v1.projects.work-logs.index
POST	api/v1/projects/{project}/work-logs	v1.projects.work-logs.store
PUT	api/v1/rates/{rate}	v1.
DELETE	api/v1/rates/{rate}	v1.
GET|HEAD	api/v1/reports/portfolio	v1.reports.portfolio
GET|HEAD	api/v1/reports/project-analytics	v1.reports.project-analytics
GET|HEAD	api/v1/security	v1.

Método	URI	Nome
PUT	api/v1/security/password	v1.
PUT	api/v1/security/preferences	v1.
DELETE	api/v1/security/providers/ {provider}	v1.
DELETE	api/v1/security/sessions	v1.
DELETE	api/v1/security/sessions/ {session}	v1.
DELETE	api/v1/security/two-factor	v1.
POST	api/v1/security/two-factor/ confirm	v1.
POST	api/v1/security/two-factor/setup	v1.
GET|HEAD	api/v1/tasks	v1.tasks.index
POST	api/v1/tasks	v1.tasks.store
PUT|PATCH	api/v1/tasks/{task}	v1.tasks.update
DELETE	api/v1/tasks/{task}	v1.tasks.destroy
PATCH	api/v1/tasks/{task}/status	v1.
GET|HEAD	api/v1/teams	v1.teams.index
GET|HEAD	api/v1/teams/{project}	v1.teams.show
PUT	api/v1/teams/{project}	v1.teams.update
GET|HEAD	auth/google	auth.google
GET|HEAD	auth/google/callback	auth.google.callback
GET|HEAD	auth/microsoft	auth.microsoft
GET|HEAD	auth/microsoft/callback	auth.microsoft.callback
GET|HEAD	first-access/{token?}	—
GET|HEAD	forgot-password	password.request
POST	forgot-password	password.email
PATCH	profile	profile.update
POST	profile/avatar	profile.avatar.update
DELETE	profile/avatar	profile.avatar.destroy
POST	profile/avatar/microsoft	—
POST	reset-password	password.update
GET|HEAD	reset-password/{token}	password.reset

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).

APÊNDICE B — EVOLUÇÃO DO ESQUEMA E ARTEFATOS

As migrations abaixo registram a evolução disponível na revisão; a ordem é a do nome dos arquivos, e não uma afirmação de que todas foram introduzidas na mesma etapa. A criação de uma coluna não prova a homologação do recurso correspondente.

As informações correspondentes estão organizadas no Quadro 7.

Quadro 7 — Migrations disponíveis

Arquivo
0001_01_01_000000_create_users_table.php
0001_01_01_000001_create_cache_table.php
0001_01_01_000002_create_jobs_table.php
2026_08_06_000000_add_google_auth_fields_to_users_table.php
2026_08_09_000000_add_microsoft_id_to_users_table.php
2026_08_09_010000_create_spot_domain_tables.php
2026_08_09_020000_expand_user_avatar_url.php
2026_08_20_000000_create_inventory_tables.php
2026_08_20_010000_create_project_management_tables.php
2026_08_23_161756_create_personal_access_tokens_table.php
2026_08_23_170000_create_project_governance_tables.php
2026_08_23_200000_align_project_membership_roles_and_lifecycle.php
2026_08_23_210000_add_timezone_to_users_table.php
2026_08_23_220000_add_administrative_identity_fields.php
2026_08_23_230000_add_account_security_fields.php
2026_08_23_240000_add_notification_preferences_to_users.php
2026_08_23_250000_create_inbox_items_table.php
2026_08_23_260000_create_calendar_integrations.php
2026_09_13_235000_create_project_documents_table.php
2026_09_16_150000_add_activity_fields_to_tasks.php
2026_09_16_160000_add_memorial_configuration.php
2026_09_16_170000_add_expense_analyst.php
2026_09_18_151700_add_password_requirement_to_project_invitations.php
2026_09_18_160500_add_team_name_to_projects.php
2026_10_26_150000_create_task_participants_table.php
2026_10_26_160000_create_notification_delivery_tables.php
2026_10_28_170000_add_task_to_project_documents.php
2026_11_05_160000_create_project_access_requests_table.php
2026_11_06_000000_unify_manager_profile_labels.php
2026_11_06_120000_create_chat_tables.php
2026_11_06_150000_create_chat_attachments_table.php
2026_11_06_170000_add_chat_message_format.php

Arquivo
2026_11_06_180000_add_chat_image_descriptions.php
2026_11_07_120000_add_notification_access_to_users.php
2026_11_07_140000_add_calendar_event_details.php
2026_11_07_160000_create_calendar_deliveries_table.php
2026_11_07_180000_add_calendar_event_groups.php
2026_11_09_120000_add_preferred_locale_to_users.php
2026_11_10_130000_add_assigned_reviewer_to_chat_requests.php

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).

As informações correspondentes estão organizadas no Quadro 8.

Quadro 8 — Documentos operacionais complementares

Arquivo	Finalidade
docs/chat.md	Regras e testes do chat.
docs/padrao-emails.md	Modelos e prévias de mensagens.
docs/acessibilidade-e-inclusao.md	Escopo e limites das verificações.
docs/revisao-producao-2026-11-10.md	Evidência e condições para homologação.
docs/implantacao-docker.md	Procedimento de implantação.
docs/fontes-documentais/manifesto-versao-final.json	Commit, hashes e fontes desta edição.

Fonte: Elaboração própria com base no código-fonte do SPOT (Longo *et al.*, 2026c).